

VIETNAM NATIONAL UNIVERSITY-HO CHI MINH CITY

HO CHI MINH UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

---

# Final Report

## Local Representation of Facial Features

---

Course: Pattern Recognition

*Student Information:*

23127004 - Le Nhat Khoi

23127165 - Nguyen Hai Dang

23127181 - Nguyen Tran Quoc Duy

*Instructors:*

Mr. Le Hoang Thai

Mr. Vo Thanh Lam

Mr. Nguyen Thanh Tinh

April 20, 2026



# Contents

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>CHAPTER 0: OVERALL SUMMARY</b>                               | <b>5</b>  |
| <b>CHAPTER 1: INTRODUCTION</b>                                  | <b>6</b>  |
| 1.1 Facial Representation                                       | 6         |
| 1.2 Evolution of Techniques                                     | 6         |
| 1.3 An Application to Deepfake Detection                        | 6         |
| <b>CHAPTER 2: LOCAL REPRESENTATION TECHNIQUE</b>                | <b>7</b>  |
| <b>2.1 Local Binary Patterns (LBP)</b>                          | <b>7</b>  |
| 2.1.1 Variation of LBP Operator . . . . .                       | 7         |
| 2.1.1.1 Basic LBP Operator . . . . .                            | 7         |
| 2.1.1.2 Spatial LBP . . . . .                                   | 8         |
| 2.1.1.3 Spatiotemporal LBP . . . . .                            | 9         |
| 2.1.1.4 Multi-resolution LBP . . . . .                          | 10        |
| 2.1.2 Face Description with LBP . . . . .                       | 11        |
| 2.1.3 Some Application with LBP Descriptor . . . . .            | 11        |
| 2.1.3.1 Face Recognition . . . . .                              | 11        |
| 2.1.3.2 Face Verification . . . . .                             | 12        |
| 2.1.3.3 Facial Expression Recognition . . . . .                 | 12        |
| 2.1.3.4 Face Detection . . . . .                                | 13        |
| 2.1.3.5 Other Face-Related Applications . . . . .               | 13        |
| <b>2.2 Gabor Feature</b>                                        | <b>13</b> |
| 2.2.1 1D Gabor Function . . . . .                               | 13        |
| 2.2.2 From 1D to 2D . . . . .                                   | 14        |
| 2.2.3 Gabor Feature Extraction . . . . .                        | 14        |
| 2.2.4 Application . . . . .                                     | 15        |
| 2.2.4.1 Facial Landmark Detection . . . . .                     | 15        |
| 2.2.4.2 Face Recognition (Identification) . . . . .             | 16        |
| <b>2.3 Final Conclusion</b>                                     | <b>16</b> |
| <b>CHAPTER 3: DEEP LEARNING ERA &amp; DEEPFAKE DETECTION</b>    | <b>17</b> |
| <b>3.1 Deep Learning Transition</b>                             | <b>17</b> |
| <b>3.2 Deepfake Detection Problem &amp; Previous Approaches</b> | <b>17</b> |
| <b>CHAPTER 4: BINARY NEURAL NETWORK</b>                         | <b>18</b> |
| <b>4.1 From Full-Precision CNNs to BNNs</b>                     | <b>18</b> |
| <b>4.2 Computational Advantages</b>                             | <b>18</b> |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>CHAPTER 5: PAPER METHODOLOGY</b>                                     | <b>19</b> |
| <b>5.1 Paper’s Position</b>                                             | <b>19</b> |
| <b>5.2 Methodology &amp; Analysis</b>                                   | <b>19</b> |
| 5.2.1 Augmented Features . . . . .                                      | 20        |
| 5.2.1.1 Theoretical Idea . . . . .                                      | 20        |
| 5.2.1.2 Implementation . . . . .                                        | 20        |
| 5.2.2 Adapter Module . . . . .                                          | 21        |
| 5.2.2.1 Theoretical Idea . . . . .                                      | 21        |
| 5.2.2.2 Implementation . . . . .                                        | 21        |
| 5.2.3 Binary Backbone . . . . .                                         | 21        |
| 5.2.3.1 Theoretical Idea . . . . .                                      | 21        |
| 5.2.3.2 Implementation . . . . .                                        | 22        |
| 5.2.4 Classification Head . . . . .                                     | 22        |
| 5.2.5 Loss Function . . . . .                                           | 23        |
| <b>CHAPTER 6: EXPERIMENT</b>                                            | <b>24</b> |
| <b>6.1 Experimental Benchmark</b>                                       | <b>24</b> |
| 6.1.1 CIFAKE Datasets . . . . .                                         | 24        |
| 6.1.1.1 Overall Information . . . . .                                   | 24        |
| 6.1.1.2 Preprocessing . . . . .                                         | 24        |
| 6.1.2 COCOFake Datasets . . . . .                                       | 25        |
| 6.1.2.1 Overall Information . . . . .                                   | 25        |
| 6.1.2.2 Preprocessing . . . . .                                         | 25        |
| <b>6.2 Experimental Setup</b>                                           | <b>26</b> |
| <b>6.3 Results &amp; Comparison</b>                                     | <b>26</b> |
| <b>6.4 Experimental Result Analysis</b>                                 | <b>27</b> |
| <b>CHAPTER 7: EXTENSION &amp; IMPROVEMENT</b>                           | <b>28</b> |
| <b>7.1 Motivation</b>                                                   | <b>28</b> |
| <b>7.2 Proposed Architecture: MobileNet4DFR</b>                         | <b>28</b> |
| 7.2.1 Backbone Replacement with MobileNetV3-Small . . . . .             | 29        |
| 7.2.2 Enhanced Adapter with Squeeze-and-Excitation (SE) Block . . . . . | 29        |
| 7.2.3 Gabor Filter Channel Augmentation . . . . .                       | 30        |
| <b>7.3 Ablation Study Analysis &amp; Discussion</b>                     | <b>31</b> |
| <b>7.4 Experimental Comparison</b>                                      | <b>32</b> |
| <b>CHAPTER 8: CONCLUSION</b>                                            | <b>33</b> |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>8.1 Derivation from Course Theory</b>                        | <b>33</b> |
| <b>8.2 Methodological Differences from Classical Approaches</b> | <b>33</b> |
| <b>8.3 Technical Strengths</b>                                  | <b>33</b> |
| <b>8.4 Limitations &amp; Discrepancies</b>                      | <b>34</b> |
| <b>8.5 Personal Technical Insights &amp; Conclusion</b>         | <b>34</b> |

## CONTRIBUTION

| Student ID | Full Name            | Assigned Responsibilities                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Rate |
|------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| 23127004   | Le Nhat Khoi         | <ul style="list-style-type: none"> <li>- <b>Theoretical Foundation:</b> Drafted Chapter 1 and Section 2.1 (LBP algorithm, spatial/spatiotemporal variations, and mathematical formulas).</li> <li>- <b>Paper Input Analysis:</b> Drafted Sections 5.1 &amp; 5.2.1, explaining the extraction of RGB, FFT Magnitude, and LBP channels in the original paper.</li> <li>- <b>Baseline Experiments:</b> Conducted the experimental setup, data preprocessing, and baseline reproduction for CIFAKE and COCOFake datasets (Chapter 6).</li> <li>- <b>Synthesis:</b> Authored Section 8.1, linking the course theory to the paper's feature augmentation approach.</li> </ul>                | 100% |
| 23127165   | Nguyen Hai Dang      | <ul style="list-style-type: none"> <li>- <b>Extended Features &amp; BNN Theory:</b> Drafted Sections 2.2, 2.3, 3, and 4. Explained Gabor features, the deep learning transition, and the XNOR/POP-COUNT logic in Binary Neural Networks.</li> <li>- <b>Architectural Improvement:</b> Proposed and implemented the MobileNet4DFR architecture, specifically replacing the backbone with MobileNetV3-Small and integrating the SE-Block Adapter (Sections 7.1, 7.2.1, 7.2.2).</li> <li>- <b>Synthesis:</b> Authored Sections 8.3 &amp; 8.4, rigorously evaluating the technical strengths, limitations, and practical hardware discrepancies of the paper's method.</li> </ul>          | 100% |
| 23127181   | Nguyen Tran Quoc Duy | <ul style="list-style-type: none"> <li>- <b>Paper Pipeline Analysis:</b> Drafted Sections 2.1.2, 2.1.3, and 5.2.2 to 5.2.5. Analyzed the traditional LBP applications, the original Adapter module, BNext backbone, and Classification Head.</li> <li>- <b>Feature Augmentation &amp; Ablation:</b> Implemented the Gabor Filter channel and conducted the comprehensive ablation study. Authored Sections 7.2.3, 7.2.4, and the final Experimental Comparison (Section 7.3).</li> <li>- <b>Synthesis:</b> Authored Sections 8.2 &amp; 8.5, highlighting the methodological differences from classical approaches and summarizing the project's overall technical insights.</li> </ul> | 100% |

# CHAPTER 0

## OVERALL SUMMARY

This report examines the local representation of facial features, tracing the evolution from classical handcrafted descriptors to modern deep learning approaches, with a concrete case study in real-time deepfake detection.

### Report Structure at a Glance

| Chapter | Title                              | Key Content                                                                                                  |
|---------|------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1       | Introduction                       | Overview of facial representation, evolution of techniques, and motivation for deepfake detection            |
| 2       | Local Representation Techniques    | Local Binary Patterns (LBP) and Gabor filters; variants, properties, and applications in texture analysis    |
| 3       | Deep Learning & Deepfake Detection | Limitations of handcrafted features; deep learning approaches and problem formulation for deepfake detection |
| 4       | Binary Neural Networks             | Lightweight deep learning models; principles of BNNs and their efficiency advantages                         |
| 5       | Paper Methodology                  | Detailed breakdown of the proposed pipeline, feature augmentation (FFT, LBP), and adapter mechanism          |
| 6       | Experiments                        | Datasets (CIFAKE, COCOFake), preprocessing, training setup, and quantitative evaluation results              |
| 7       | Extension & Improvement            | Proposed MobileNet4DFR architecture, SE-enhanced adapter, and Gabor feature integration                      |
| 8       | Conclusion                         | Summary of findings, connections to theory, limitations, and key insights                                    |

### Research Paper

- **Paper:** *“Faster Than Lies: Real-time Deepfake Detection using Binary Neural Networks”*
- **Authors:** Romeo Lanzino, Federico Fontana, Anxhelo Diko, Marco Raoul Marini, Luigi Cinque (Sapienza University of Rome)
- **Conference:** The IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW 2024)
- **Rank:** A\* (CORE ranking — top tier in Computer Vision)
- **Code:** [https://github.com/fedeloper/binary\\_deepfake\\_detection](https://github.com/fedeloper/binary_deepfake_detection)

# CHAPTER 1

## INTRODUCTION

### 1.1 Facial Representation

Developing a robust face recognition system is based on two fundamental pillars: Facial Representation and Classifier Design.

While the classifier is responsible for making the final decision, the representation stage is arguably more critical. Its primary objective is to transform raw pixel data by computationally encoding into a meaningful set of features. The representation must minimize intra-class variations and while simultaneously maximizing extra-class variations.

### 1.2 Evolution of Techniques

However, finding a perfect feature set is challenging due to the immense variability in human facial appearance. Factors such as lighting conditions, camera pose, facial expressions, aging, and accessories (like glasses) create significant noise in the data.

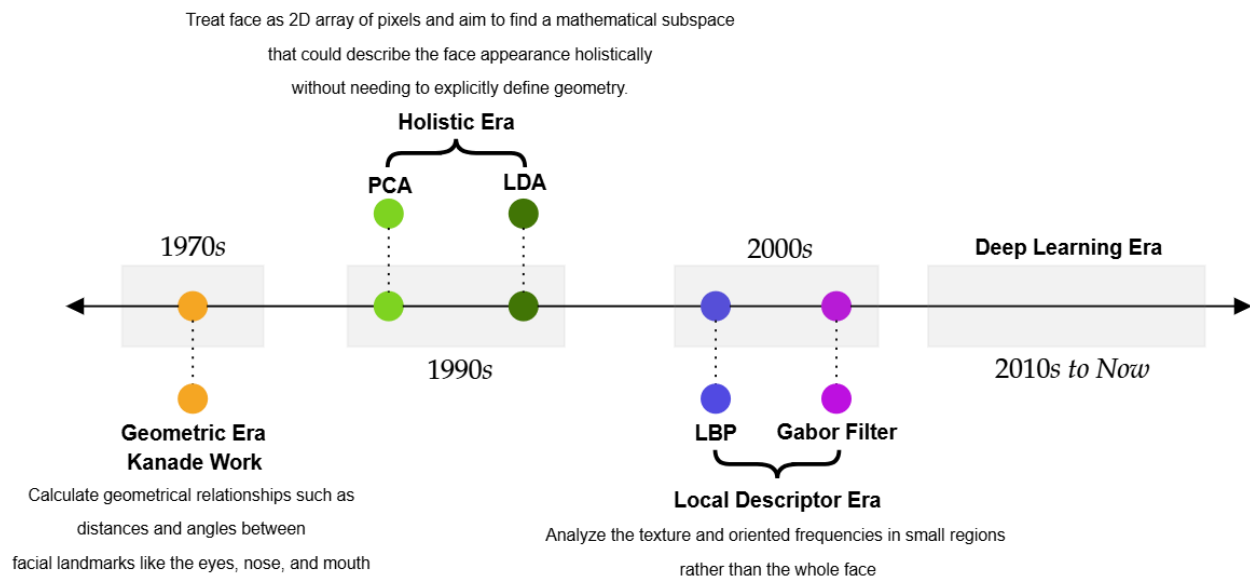


Figure 1: Technique development timeline

### 1.3 An Application to Deepfake Detection

The rapid progress of deep generative models now can produce highly realistic facial content known as DeepFake. However, they introduce serious concerns related to misinformation and privacy.

Traditional handcrafted descriptors such as Local Binary Patterns (LBP) and Gabor filters are effective at capturing local texture information; however, modern deepfakes often contain subtle artifacts that challenge shallow representations. Consequently, recent research has shifted toward deep learning-based solutions. In this context, the paper "*Faster Than Lies: Real-time Deepfake Detection using Binary Neural Networks*" proposes an efficient lightweight approach that enables accurate real-time deepfake detection on resource-constrained devices.

## CHAPTER 2

### LOCAL REPRESENTATION TECHNIQUE

#### 2.1 Local Binary Patterns (LBP)

The Local Binary Pattern (LBP) is a highly popular local texture descriptor in computer vision. The core concept of LBP involves encoding the micro-structures of an image by comparing the intensity of a central pixel with its surrounding neighbors.

Due to its computational simplicity, high efficiency, and robustness to monotonic gray-scale transformations, LBP has been adopted in tasks such as face recognition, texture classification, and video analysis.

A typical LBP-based pipeline consists of four main stages: LBP Encoding, Histogram Construction, Feature Vector Formation and Similarity Measurement/Classification

##### 2.1.1 Variation of LBP Operator

###### 2.1.1.1 Basic LBP Operator

In the original version, the LBP is calculated within a  $3 \times 3$  window. The LBP code at the center pixel is defined as:

$$LBP(x, y) = \sum_{p=0}^7 \text{sgn}(I_p - I_c) 2^p$$

where  $I_c$  be the gray value of the center pixel and  $I_p$  be the gray value of the  $p^{th}$  neighbour.

In fact, the final formula can be broken down into steps as the illustration:

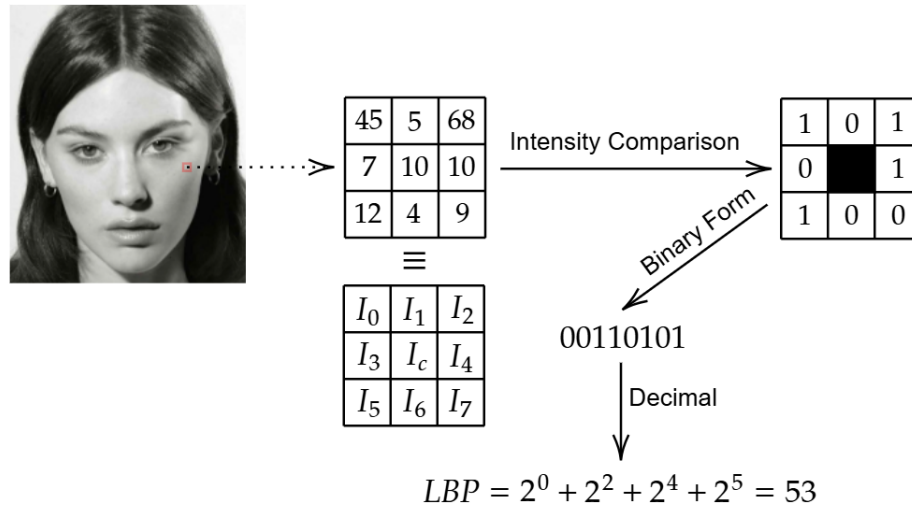


Figure 2: Step by step LBP illustration

After computing the LBP code for every pixel, we obtain an LBP label image. The histogram is defined as:

$$H_i = \sum_{x,y} \mathbf{1}\{LBP(x, y) = i\}, i = \overline{0, n-1}$$

where  $n$  is the number of different labels produced by the LBP operator

Finally, when the image patches whose histograms are to be compared have different sizes, the histograms must be normalized:

$$N_i = \frac{H_i}{\sum_{j=0}^{n-1} H_j}$$

### 2.1.1.2 Spatial LBP

The original LBP operator is limited in spatial support and produces high-dimensional histograms when generalized. Several spatial variants have been proposed to improve flexibility and robustness including:

- **Circular LBP:** Circular LBP is parameterized by:
  - $P$  is the number of sampling points on the circle
  - $R$  is the radius of the circle

As illustration, for a center pixel  $(x, y)$ , the neighbors are sampled on a circle of radius  $R$ :

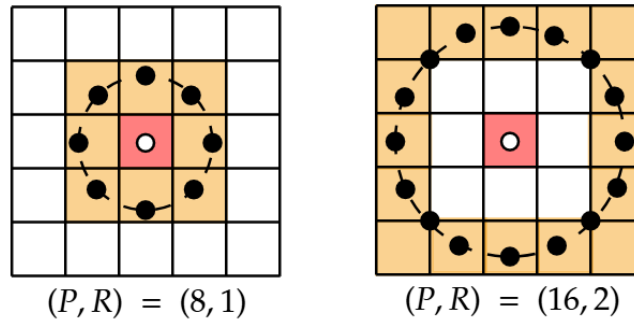


Figure 3: Circular LBP

In mathematical expression, the sampling points are located at:

$$(x_p, y_p) = (x + R \cos \frac{2\pi p}{P}, y - R \sin \frac{2\pi p}{P})$$

which requires Bilinear interpolation to estimate the gray value  $I_p$

However, the number of possible labels grows exponentially  $2^P$  which will lead to sparse histograms and unstable statistics.

- **Uniform LBP:** Uniform LBP is inspired by the fact that some binary patterns occur more frequently than others in texture images.

A binary pattern is called uniform if the circular bit string contains at most two transitions between 0 and 1. In mathematical expression, we can state:

$$\mathbf{b} \text{ is uniform} \iff U(\mathbf{b}) = \sum_{p=0}^{P-1} |\mathbf{b}_p - \mathbf{b}_{(p+1) \bmod P}| \leq 2$$

For example, you can calculate and identify whether the binary pattern is uniform or not:

$$\begin{array}{ll}
 \mathbf{b}^{(1)} = 01110000 \longrightarrow U(\mathbf{b}^{(1)}) = 2 & \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{Uniform} \\
 \mathbf{b}^{(2)} = 11001111 \longrightarrow U(\mathbf{b}^{(2)}) = 2 & \\
 \mathbf{b}^{(3)} = 11001001 \longrightarrow U(\mathbf{b}^{(3)}) = 4 & \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{Non-uniform} \\
 \mathbf{b}^{(4)} = 01010011 \longrightarrow U(\mathbf{b}^{(4)}) = 6 &
 \end{array}$$

Figure 4: Example illustration

In labeling strategy, each uniform pattern receives its own label and all non-uniform patterns are grouped into one bin. As a result, for  $P = 8$ , there are a total of 256 patterns of which 58 are uniform thus yielding to the total of 59 different labels.

In conclusion, the combination of these two extension, Uniform Circular LBP is commonly denoted  $LBP_{P,R}^{u_2}$  where  $u_2$  means uniform with at most 2 transitions.

### 2.1.1.3 Spatiotemporal LBP

In many real-world problems, especially facial expression analysis and dynamic texture recognition require modeling temporal variations. To address this limitation, LBP was extended into the spatiotemporal domain includes:

- **Volume LBP (VLBP):** This simply treats a video as a 3D volume in  $(X, Y, T)$  where:
  - $X, Y$  are the spatial coordinates
  - $T$  is the time coordinates (in other word, frame index)

Instead of analyzing a 2D neighborhood, VLBP analyzes a 3D neighborhood around each pixel in space-time. Here is the visualization:

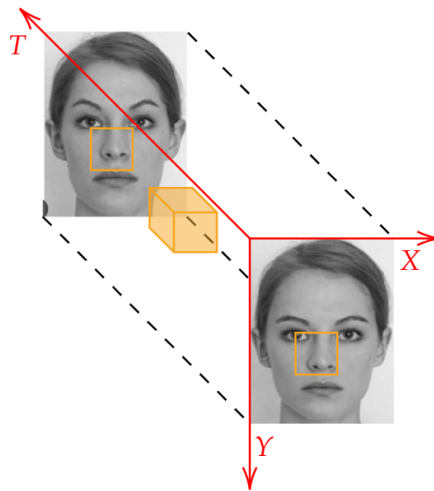


Figure 5: Volume LBP visualization

However, VLBP suffers from combinatorial explosion results in high-dimensional vectors and poor real-time suitability.

- **LBP-TOP (Three Orthogonal Planes):** To make VLBP more computationally efficient, LBP-TOP computes standard 2D LBP on three orthogonal planes through the video volume:

- **XY plane (Spatial Appearance):**  $LBP_{P_{XY}, R_{XY}}^{u_2}(x, y, t) \rightarrow H_{XY} \rightarrow N_{XY}$
- **XT plane (Horizontal Motion):**  $LBP_{P_{XT}, R_{XT}}^{u_2}(x, y, t) \rightarrow H_{XT} \rightarrow N_{XT}$
- **YT plane (Vertical Motion):**  $LBP_{P_{YT}, R_{YT}}^{u_2}(x, y, t) \rightarrow H_{YT} \rightarrow N_{YT}$

where  $H_{\_}$  and  $N_{\_}$  are the histogram and normalized histogram of the corresponding plane, respectively. Then the final feature vector is generated by concatenating:

$$N = [N_{XY} \| N_{XT} \| N_{YT}]$$

#### 2.1.1.4 Multi-resolution LBP

Single-scale LBP cannot simultaneously capture fine micro-textures or large micro-textures. Multi-resolution LBP has been proposed to fix this including:

- **Multi-scale LBP:** The approach is simple, Multiple LBP operators with different  $(P, R)$  will be applied. Each pixel is encoded at several scales:

$$LBP_{P_1, R_1}, LBP_{P_2, R_2}, \dots$$

Instead of building a joint histogram (which would be extremely sparse), separate histograms are computed for each scale and then concatenated.

- **Multi-scale Block LBP (MB-LBP):** MB-LBP extends LBP by comparing average intensities of blocks instead of individual pixels as illustration:

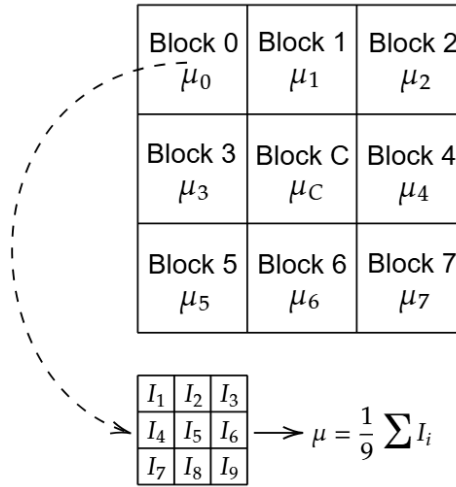


Figure 6: Multi-Scale Block LBP illustration

The procedure consists:

- Dividing neighborhood into sub-blocks.
- Computing mean intensity of each block.
- Applying the LBP comparison on block means.

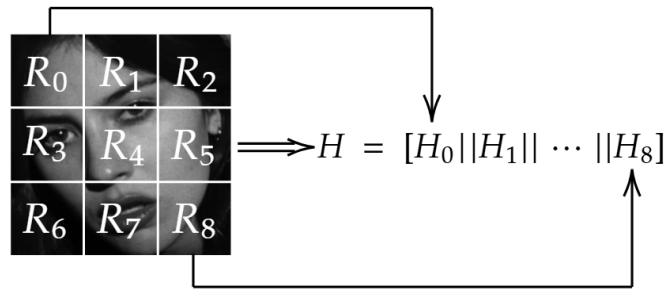
As a result, if we use  $3 \times 3$  blocks, effective support may cover a  $9 \times 9$  area.

### 2.1.2 Face Description with LBP

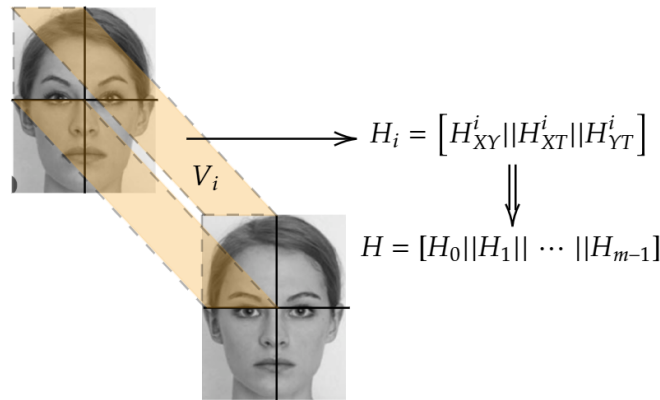
In classical texture classification, the occurrences of Local Binary Pattern (LBP) codes are typically accumulated into a single global histogram, and classification is performed by comparing histogram similarities.

However, applying this holistic strategy directly to facial images will make global histogram ignores spatial information.

Therefore, instead, the face image is divided into  $m$  local regions:  $R_0, R_1, \dots, R_{m-1}$  which may do not have to be a rectangle and can be overlapped and varied in sizes. At each region, we then calculate LBP as well as their region histogram then simply concatenate them:



In the case of sequences of face, we perform the same method by divided into block volumes:  $V_1, V_2, \dots, V_{m-1}$  with block histogram  $H_i$  make from plane histograms  $H_i = [H_{XY}^i || H_{XT}^i || H_{YT}^i]$ . Finally, we simply concatenate them as the way we do in static image:



### 2.1.3 Some Application with LBP Descriptor

Local Binary Pattern (LBP) descriptors have been successfully applied to a wide range of face analysis problems. This section will introduce some applications with LBP descriptor we present in the previous part.

#### 2.1.3.1 Face Recognition

In face recognition, the goal is to identify the identity of a person from a facial image. After extracting the spatially enhanced LBP descriptor described in the previous section, recognition is typically performed using a distance-based classifier such as the nearest neighbor (NN) rule.

Let  $\mathbf{x}$  and  $\boldsymbol{\xi}$  denote two normalized enhanced histograms. The similarity between two faces is commonly measured using the Chi-square distance:

$$\chi^2(\mathbf{x}, \boldsymbol{\xi}) = \sum_{j=0}^{m-1} \sum_{i=0}^{n-1} \frac{(x_{i,j} - \xi_{i,j})^2}{x_{i,j} + \xi_{i,j}}$$

where  $i$  and  $j$  indexes the histogram bin and local facial region with  $m$  and  $n$  is the number of regions and LBP labels, respectively

Since different facial regions contribute unequally to recognition, a weighted version is used:

$$\chi_w^2(\mathbf{x}, \boldsymbol{\xi}) = \sum_{j=0}^{m-1} w_j \sum_{i=0}^{n-1} \frac{(x_{i,j} - \xi_{i,j})^2}{x_{i,j} + \xi_{i,j}}$$

where  $w_j$  is the importance weight of region  $j$ .

The identity of a probe face is then determined by the nearest neighbor rule:

$$\hat{y} = \arg \min_k \chi_w^2(\mathbf{x}, \mathbf{g}_k)$$

where  $\mathbf{g}_k$  is the gallery descriptor of subject  $k$ .

There are also many other extensions and improvements including:

- **Tan–Triggs Preprocessing:** To further improve robustness under severe lighting, this was proposed that normalizes illumination before LBP extraction.
- **Multi-scale LBP with LDA:** Extracting multi-scale LBP histograms and projecting the concatenated feature vector into a discriminative subspace using LDA.
- **AdaBoost-based Region Selection:** AdaBoost is used to select the most informative facial regions and learn optimal region weights.

### 2.1.3.2 Face Verification

Face verification determines whether two facial images belong to the same person. Instead of nearest neighbor identification, verification is usually formulated as a binary hypothesis test.

In probabilistic LBP-based systems, local histograms are interpreted as empirical distributions. Given a client model  $\mathcal{M}_c$  and a generic model  $\mathcal{M}_g$ , the decision is made using the log-likelihood ratio:

$$\Lambda(\mathbf{x}) = \log \frac{p(\mathbf{x} | \mathcal{M}_c)}{p(\mathbf{x} | \mathcal{M}_g)}, \text{ accept if } \Lambda(\mathbf{x}) > \tau$$

where  $\tau$  is a predefined threshold.

### 2.1.3.3 Facial Expression Recognition

LBP and its spatiotemporal variants are widely used for facial expression analysis. Static LBP captures appearance information, while LBP-TOP encodes dynamic motion patterns from video sequences.

These descriptors are typically fed into classifiers such as:

- Support Vector Machine (SVM)
- k-Nearest Neighbor (kNN)
- Linear Discriminant Analysis (LDA)

#### 2.1.3.4 Face Detection

LBP features are also effective for real-time face detection. In this task, compact LBP variants (e.g.,  $LBP_{4,1}$ ) and overlapping regions are often used to reduce feature dimensionality and improve statistical reliability on low-resolution images.

#### 2.1.3.5 Other Face-Related Applications

Beyond the major tasks above, LBP-based descriptors have been successfully applied to:

- Gender recognition
- Age estimation
- Head pose estimation
- Iris recognition
- 3D face recognition

## 2.2 Gabor Feature

In visual analysis, we usually want to know two things at the same time: where a structure is located (spatial localization) and what frequency pattern it contains (frequency localization). However, there is a natural trade-off between these two goals:

- A pure sinusoid gives perfect frequency information, but it stretches infinitely in space, so it does not tell us where the pattern occurs.
- A Dirac delta impulse is perfectly localized in space, but it contains all frequencies equally, so it provides no frequency selectivity.

This limitation comes from the Fourier uncertainty principle. Dennis Gabor showed that a very good compromise can be achieved by multiplying a sinusoid with a Gaussian envelope.

### 2.2.1 1D Gabor Function

In one dimension, the complex Gabor function is defined as

$$\psi(t) = \exp\left(-\frac{(t - t_0)^2}{2\sigma^2}\right) \exp(j(2\pi f_0 t + \phi)).$$

where  $\alpha$  is the bandwidth of Gaussian,  $t_0$  is the time shift defining the time location of the Gaussian and  $f_0$  is the frequency of the harmonic oscillations

### 2.2.2 From 1D to 2D

Images are two-dimensional signals, so the Gabor function must be extended to 2D. The additional parameter is orientation selectivity  $\theta$ , since image structures such as edges have directions.

The 2D Gabor function becomes:

$$\psi(x, y) = \exp\left(-\left(\frac{f^2}{\gamma^2}x'^2 + \frac{f^2}{\eta^2}y'^2\right)\right) \exp(j2\pi f x'),$$

where the rotated coordinates are

$$x' = x \cos \theta + y \sin \theta, \quad y' = -x \sin \theta + y \cos \theta.$$

### 2.2.3 Gabor Feature Extraction

Given an input image  $\xi(x, y)$  and a Gabor filter  $\psi(x, y; f, \theta)$ , the filter response is computed by convolution:

$$r_\xi(x, y; f, \theta) = \psi(x, y; f, \theta) * \xi(x, y).$$

At each spatial location, the complex response measures how strongly the local image pattern matches the filter's preferred frequency  $f$  and orientation  $\theta$ .

However, a single Gabor filter captures only one frequency and one direction. To represent natural images effectively, a filter bank is constructed by sampling multiple frequencies and orientations.

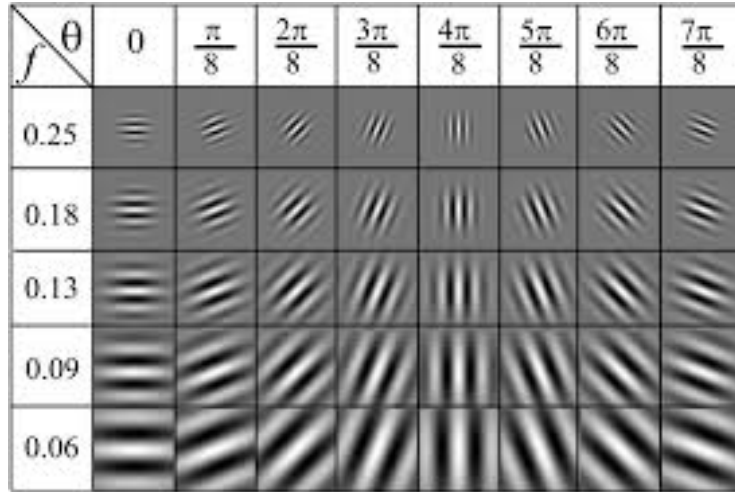


Figure 7: Gabor Bank visualization from [Research Gate](#)

- Frequencies are typically sampled on a logarithmic scale:

$$f_k = c^{-k} f_{\max}, \quad k = 0, \dots, m-1,$$

where  $c > 1$  is the scaling factor.

- Orientations are uniformly spaced:

$$\theta_\ell = \frac{\ell\pi}{n}, \quad \ell = 0, \dots, n-1.$$

At a fixed spatial location  $(x_0, y_0)$ , the responses from all filters are collected to form a feature matrix:

$$G(x_0, y_0) = \begin{bmatrix} r(f_0, \theta_0) & \cdots & r(f_0, \theta_{n-1}) \\ \vdots & \ddots & \vdots \\ r(f_{m-1}, \theta_0) & \cdots & r(f_{m-1}, \theta_{n-1}) \end{bmatrix}.$$

Rows correspond to different scales (frequencies) and columns correspond to different orientations. This local descriptor is known as a *Gabor jet*.

For machine learning applications, the matrix is typically flattened into a single feature vector

$$g(x_0, y_0) = [r(f_0, \theta_0), \cdots, r(f_{m-1}, \theta_{n-1})]$$

## 2.2.4 Application

Once Gabor features have been extracted and statistically modeled, they can be used in several face analysis tasks. The common theme is to treat the Gabor jet at each location as a discriminative local descriptor and then apply either probabilistic models or distance-based matching.

### 2.2.4.1 Facial Landmark Detection

Let each facial landmark type (left eye, right eye, nose tip, etc.) be treated as a separate class. Given an image location  $(x, y)$  with Gabor feature vector  $\mathbf{g}(x, y)$ , we evaluate how likely this location belongs to each landmark class using

$$p(\mathbf{g}(x, y) \mid \text{class}).$$

The detected landmark position is the location with the highest likelihood.

The full pipeline is illustrated in the following image:

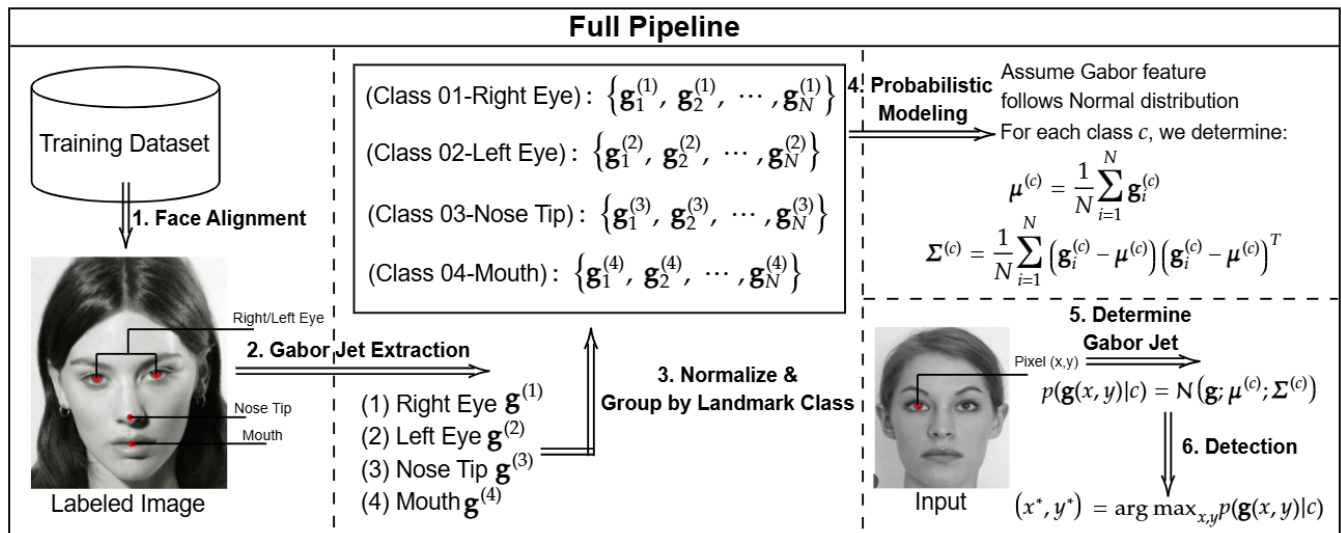


Figure 8: Full Pipeline of Facial Landmark Detection

### 2.2.4.2 Face Recognition (Identification)

After detecting and aligning the face, Gabor jets are extracted at a set of stable facial landmarks. These local descriptors are concatenated into a global face feature vector.

Recognition is then performed by comparing the probe face against a gallery of known identities using Distance Measures or Likelihood under class models.

## 2.3 Final Conclusion

Local feature descriptors are fundamental to robust face representation under variations in illumination, pose, and imaging conditions. Among them, Local Binary Patterns (LBP) and Gabor-based features remain two of the most influential approaches.

| Property                | LBP                                      | Gabor Filter                       |
|-------------------------|------------------------------------------|------------------------------------|
| Primary information     | Local texture / intensity ordering       | Frequency & orientation content    |
| Computational cost      | Very low (integer ops)                   | High (complex convolutions)        |
| Illumination robustness | High (rank-order based)                  | Moderate (normalisation-dependent) |
| Frequency selectivity   | None                                     | Strong (multi-scale bank)          |
| Spatial locality        | Strong ( $3 \times 3$ to custom $P, R$ ) | Strong (Gaussian envelope)         |
| Typical use             | Real-time texture classification         | Precision face analysis            |

**LBP** offers fast and illumination-robust texture encoding by quantizing local intensity ordering. Its main limitation is sensitivity to noise and flat regions, which can lead to unstable codes. Numerous variants—such as LTP, Dominant LBP, and LDP—have been proposed to improve robustness. LBP has also inspired related descriptors like WLD and LPQ, the latter showing strong blur robustness through local phase quantization.

In contrast, **Gabor filters** provide rich frequency and orientation selectivity, making them highly effective for detailed facial analysis. Many studies demonstrate that combining LBP and Gabor features yields superior recognition performance, although at the cost of very high feature dimensionality that often requires techniques such as Fisher Discriminant Analysis.

Overall, LBP emphasizes computational efficiency and illumination invariance, whereas Gabor features capture discriminative spectral structure. Their complementary nature has driven the development of hybrid representations that bridge classical handcrafted descriptors and modern efficient learning-based models.

This complementarity is not merely theoretical: the research paper studied in Chapters 5–6 directly exploits it by combining an **LBP channel** (spatial texture) and a **FFT magnitude channel** (global spectral fingerprint—a computationally efficient proxy for the Gabor bank) as input to a Binary Neural Network. Their ablation study confirms that **FFT + LBP together (91.60%) outperform either alone** and the RGB baseline (90.35%), validating the complementarity prediction from this chapter.

There is also a deeper conceptual resonance worth noting: LBP binarises pixel-neighbourhood relationships using the sign function; Binary Neural Networks binarise the *entire network's* weights and activations using the same sign function. This echo - from the classical binary texture descriptor of the early 2000s to modern binary deep networks - forms the unifying thread of this report.

## CHAPTER 3

### DEEP LEARNING ERA & DEEPPFAKE DETECTION

#### 3.1 Deep Learning Transition

**Motivation:** The limitations of handcrafted facial descriptors have motivated the research community to shift toward deep learning-based solutions. Traditional features such as LBP or Gabor are efficient and interpretable, but they often struggle to capture highly complex and nonlinear visual patterns present in modern face analysis tasks.

**Dominance of Deep Learning:** With the rapid growth of computational power and large-scale datasets, Convolutional Neural Networks (CNNs) have become the dominant paradigm for learning robust facial representations directly from data.

**An Effort of Building Lightweight Deep Model:** However, many state-of-the-art approaches rely on large and computationally expensive architectures, which limits their deployment in real-time or resource-constrained environments. This challenge motivates recent research toward lightweight yet accurate deep learning models.

#### 3.2 Deepfake Detection Problem & Previous Approaches

Formally, given an input image  $\mathbf{x} \in \mathbb{R}^{3 \times H \times W}$ , the goal of deepfake detection is to learn a classifier

$$f_{\theta}(\mathbf{x}) \rightarrow \{0, 1\}$$

where 0 and 1 denote real and fake images, respectively.

The classifier  $f_{\theta}$  is typically implemented using deep neural networks parameterized by  $\theta$ , and trained on a labeled dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N,$$

where  $y_i \in \{0, 1\}$  indicates whether the image is authentic or manipulated. The learning objective is commonly formulated as a binary classification problem using the cross-entropy loss. There are several previous approaches to this problem includes:

- **Artifact-based Methods:** Early detectors focused on hand-crafted visual artifacts such as frequency irregularities, color inconsistencies, or physiological cues. While computationally efficient, these methods often overfit to specific manipulation pipelines and generalize poorly to unseen deepfakes.
- **CNN-based Approaches:** To enhance robustness, later works adopted convolutional neural networks that automatically learn discriminative features from raw images. Their ability to capture complex spatial patterns has made CNN-based detectors the dominant approach in modern deepfake detection.

**Limitation** Despite strong performance, most high-accuracy detectors rely on large backbone networks with millions of parameters and high floating-point computational cost (FLOPs) making it difficult deploying real-time system

→ We need to find a more efficient and real-time approach

## CHAPTER 4

### BINARY NEURAL NETWORK

These challenges motivate the exploration of lightweight deep learning models that preserve discriminative power while reducing computational overhead. Among various model compression strategies, Binary Neural Networks (BNNs) have emerged as an effective approach for building computationally efficient deep models. The key idea is to quantize both network weights and activations to binary values, typically in  $\{-1, +1\}$ , thereby replacing most floating-point multiplications with inexpensive bit-wise operations.

#### 4.1 From Full-Precision CNNs to BNNs

In a standard convolutional neural network, the output feature map of a convolution layer is computed as

$$\mathbf{Y} = \mathbf{A}_r \otimes \mathbf{W}_r,$$

where  $\mathbf{A}_r$  and  $\mathbf{W}_r$  denote real-valued activations and weights.

Binary Neural Networks (BNNs) approximate this computation by binarizing both operands using the sign function:

$$\mathbf{A}_b = \text{sgn}(\mathbf{A}_r), \quad \mathbf{W}_b = \text{sgn}(\mathbf{W}_r).$$

However, direct binarization causes significant information loss. The real-valued tensors are approximated with scaling factors:

$$\mathbf{A}_r \approx \alpha \mathbf{A}_b, \quad \mathbf{W}_r \approx \beta \mathbf{W}_b,$$

where

$$\alpha = \frac{1}{n} \|\mathbf{A}_r\|_1, \quad \beta = \frac{1}{n} \|\mathbf{W}_r\|_1.$$

Therefore, the convolution can be approximated as

$$\mathbf{Y} = \mathbf{A}_r \otimes \mathbf{W}_r \approx (\alpha \mathbf{A}_b) \otimes (\beta \mathbf{W}_b) = (\alpha\beta) (\mathbf{A}_b \circledast \mathbf{W}_b),$$

where  $\circledast$  denotes bit-wise XNOR followed by population count (POPCOUNT).

#### 4.2 Computational Advantages

The binarization process yields significant efficiency gains:

- multiplications  $\rightarrow$  bit-wise XNOR operations,
- floating-point accumulations  $\rightarrow$  integer POPCOUNT,
- memory usage reduced by up to 32 times.

As a result, the theoretical computational cost of binary convolution is dramatically lower than that of full-precision CNNs, making BNNs particularly attractive for real-time applications.

## CHAPTER 5

### PAPER METHODOLOGY

#### 5.1 Paper's Position

Building upon these developments, the paper *“Faster Than Lies: Real-time Deepfake Detection using Binary Neural Networks”* explores the use of a modern binary backbone (BNext) for efficient deepfake detection. The core idea is to combine:

- The efficiency of Binary Neural Networks,
- The representation power of deep CNN backbones,
- The complementary handcrafted cues (FFT and LBP).

By doing so, the authors aim to bridge the gap between **high detection accuracy** and **real-time deployability**. The method achieves state-of-the-art accuracy on three benchmark datasets (COCOFake, DFFD, CIFAKE) with **up to 20× reduction in FLOPs** during inference compared to existing approaches.

#### 5.2 Methodology & Analysis

The proposed framework aims to classify an input RGB image as either *real* or *fake*. As illustration, pipeline consists of **three** main stages:

- (i) Augmented feature construction
- (ii) Channel adaptation via an Adapter module
- (iii) Feature extraction using a binary backbone followed by Classifier

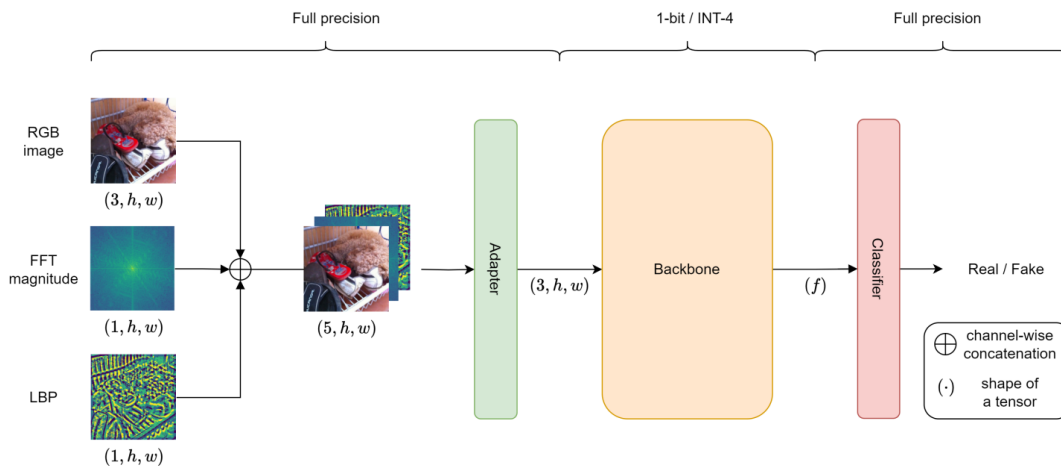


Figure 9: Overall Pipeline

## 5.2.1 Augmented Features

### 5.2.1.1 Theoretical Idea

Given an RGB image  $\mathbf{x} \in \mathbb{R}^{3 \times H \times W}$  is added with two channels with FFT magnitude and LBP.

- **FFT magnitude channel:** The RGB image is first converted to grayscale and transformed into the frequency domain using the 2D Fast Fourier Transform (FFT):

$$\mathbf{F} = |\mathcal{F}(\mathbf{x}_{gray})|.$$

To enhance the visibility of high-frequency artifacts, a logarithmic scaling is applied:

$$\mathbf{F} := \log(1 + \mathbf{F}).$$

This channel captures spectral anomalies—such as periodic patterns and frequency irregularities—often introduced by generative models. GANs and diffusion models leave characteristic frequency-domain artefacts (e.g., upsampling and smoothing effects) that are difficult to observe spatially. Hence, the FFT magnitude serves as a global spectral fingerprint of potential manipulation.

- **LBP channel:** Local Binary Pattern (LBP) is then computed in a circular neighborhood with  $P = 8$  sampling points and radius  $R = 1$ :  $LBP_{8,1}$  which is just the simple LBP operation

The final augmented input concatenates all channels:  $\mathbf{x}_{aug} = [\mathbf{x} \parallel F \parallel \text{LBP}] \in \mathbb{R}^{5 \times H \times W}$

The ablation study from the original paper confirms the complementary nature of these features:

| Configuration             | Accuracy (%) in COCOFake |
|---------------------------|--------------------------|
| Baseline (RGB only)       | 90.35                    |
| Magnitude                 | 82.36                    |
| FFT                       | 88.18                    |
| LBP                       | 88.42                    |
| Magnitude + FFT           | 81.20                    |
| Magnitude + LBP           | 81.67                    |
| <b>FFT + LBP (chosen)</b> | <b>91.60</b>             |

This suggests that frequency-domain and local texture cues capture different aspects of forgery artifacts and jointly provide a more discriminative representation.

### 5.2.1.2 Implementation

In the implementation, FFT and LBP features are computed during preprocessing from the grayscale image. FFT is obtained using *NumPy*'s Fourier transform with log-scaled magnitude, while LBP is extracted from the *scikit-image* library.

## 5.2.2 Adapter Module

### 5.2.2.1 Theoretical Idea

Since the backbone expects 3-channel input, an Adapter layer is introduced to fuse the augmented features. The Adapter is implemented as a learnable convolution:

$$\mathbf{x}_{adapt} = \phi(\mathbf{W}_a \otimes \mathbf{x}_{aug}),$$

where  $\mathbf{W}_a$  denotes the Adapter weights and  $\phi(\cdot)$  is a nonlinear activation. This operation projects the 5-channel tensor back to

$$\mathbf{x}_{adapt} \in \mathbb{R}^{3 \times H \times W},$$

while preserving the most informative cues from the augmented representation.

### 5.2.2.2 Implementation

The adapter is implemented as a single 2D convolutional layer (*nn.Conv2d*) with a  $3 \times 3$  kernel, stride 1, and padding 1, mapping the augmented multi-channel input to a 3-channel output.

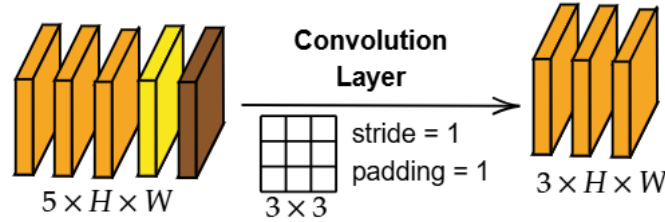


Figure 10: Detailed adapter implementation

## 5.2.3 Binary Backbone

### 5.2.3.1 Theoretical Idea

To enable real-time inference, the proposed framework adopts a **Binary Neural Network (BNN) backbone**. Specifically, the paper employs the BNext architecture which significantly reduces memory footprint and computational cost while preserving discriminative capability for deepfake detection:

In BNext, each convolution operation is **fully binary**, and other operations are **quantised to INT-4**, resulting in much more efficient networks in terms of operations. Formally, the binary convolution approximates full-precision computation as follows. Starting from the standard convolution  $Y = A_r \otimes W_r$ , BNext binarises both weights and activations using the sign function:

$$A_b = \text{sign}(A_r), \quad W_b = \text{sign}(W_r)$$

where  $\text{sign}(x_r) = +1$  if  $x_r \geq 0$ , else  $-1$ . Following XNOR-Net, a channel-wise scaling factor  $\alpha$  is introduced to reduce quantisation error. The binary convolution output becomes:

$$Y \approx (A_b \otimes W_b) \odot \alpha$$

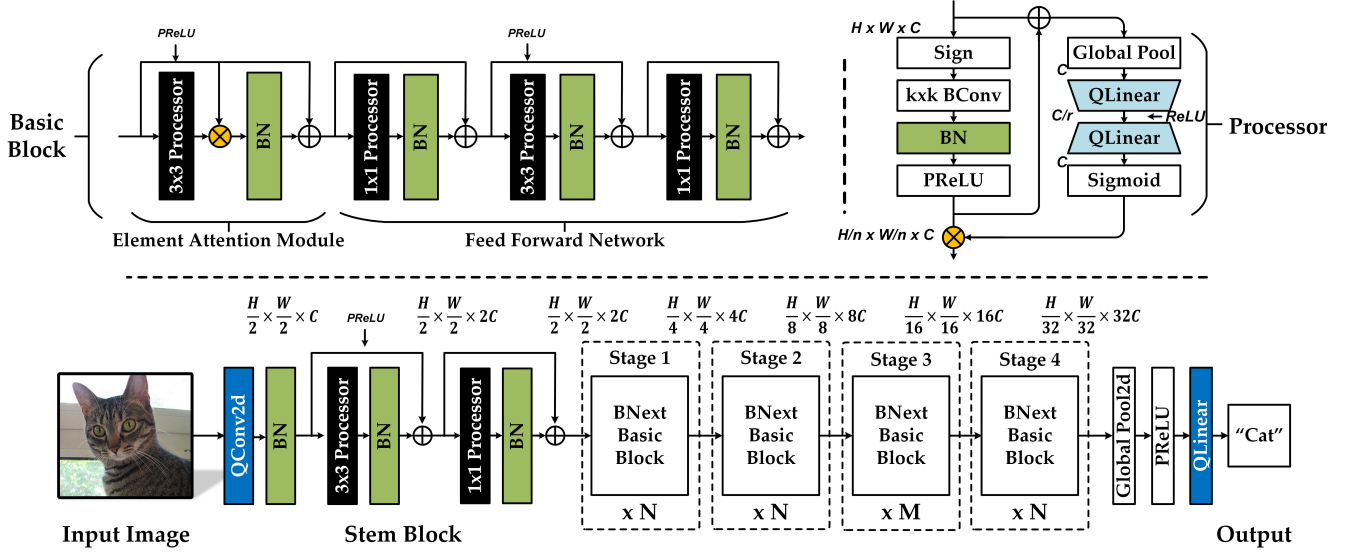


Figure 11: BNext Architecture

where  $\otimes$  denotes **XNOR-POPCOUNT** bit-wise operations and  $\odot$  is element-wise multiplication. This yields a theoretical **58 $\times$  inference speedup and 32 $\times$  memory reduction** compared to full-precision CNNs.

The backbone uses a **binary convolution module with full-precision skip-connections** and a branch with precision INT-4 to facilitate information propagation and alleviate possible bottlenecks. Three model sizes are evaluated:

- **BNext-T** (Tiny): 0.89 GFLOPs, 29.8M parameters
- **BNext-S** (Small): 1.91 GFLOPs, 67.1M parameters
- **BNext-M** (Medium): 3.39 GFLOPs, 133M parameters

### 5.2.3.2 Implementation

The backbone is implemented using the BNext architecture, initialized with pretrained weights. The model is loaded and its final classification layer is removed by replacing it with an identity mapping (`nn.Identity`), allowing the network to act as a feature extractor. The dimensionality of the extracted features is obtained from the original fully connected layer.

During training, the backbone can be optionally frozen by disabling gradient updates for its parameters. The adapted input is forwarded through the backbone to extract high-level feature representations, which are then passed to the classification head.

### 5.2.4 Classification Head

Let  $\mathbf{f} \in \{-1, +1\}^d$  denote the features extracted by the binary backbone. A full-precision linear classifier produces the logits:

$$\mathbf{z} = \mathbf{W}_c \mathbf{f} + \mathbf{b}_c,$$

followed by a sigmoid function to obtain the final prediction

$$\hat{y} = \sigma(\mathbf{z}) \in [0, 1],$$

where  $\hat{y}$  indicates the probability that the input image is fake. The model is trained with binary cross-entropy loss using the AdamW optimiser (lr= $10^{-4}$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ , weight\_decay= $10^{-2}$ ) with a learning rate scheduler that reduces to  $10^{-5}$  by the end of epoch 5.

The full-precision classification head is kept in floating point to ensure precise decision boundaries, since binarising the final logit layer would degrade accuracy without meaningful efficiency gains (it represents a negligible fraction of total operations).

### 5.2.5 Loss Function

The model is trained using Binary Cross-Entropy with logits. Given a predicted logit  $z$  and ground-truth label  $y$ , the loss is computed directly from logits for improved numerical stability.

To address class imbalance, a positive class weighting factor  $w$  is introduced:

$$\mathcal{L}(z, y) = -[w \cdot y \cdot \ell(z) + (1 - y) \cdot \ell'(z)],$$

where  $w$  is defined as the ratio between negative and positive samples in the training set.

## CHAPTER 6

### EXPERIMENT

## 6.1 Experimental Benchmark

In this section, we evaluate the proposed method on two publicly available datasets, namely **CIFAKE** and **COCOFake**. Although the original study also reports results on the DFFD dataset, we do not include it in our experiments due to accessibility and licensing constraints.

### 6.1.1 CIFAKE Datasets

#### 6.1.1.1 Overall Information

We conduct experiments on the **CIFAKE dataset**, which contains both real and AI-generated images organized into two classes: *REAL* and *FAKE*. Here are some more detailed information:

- **Number of samples:** The dataset comprises a total of 120,000 images including 60,000 real images and 60,000 fake images. The real images are collected from the CIFAR-10 dataset, while the fake images are generated using Stable Diffusion
- **Training & Testing dataset:** 100,000 training samples and 20,000 testing samples.
- **Image size:**  $32 \times 32$  RGB images

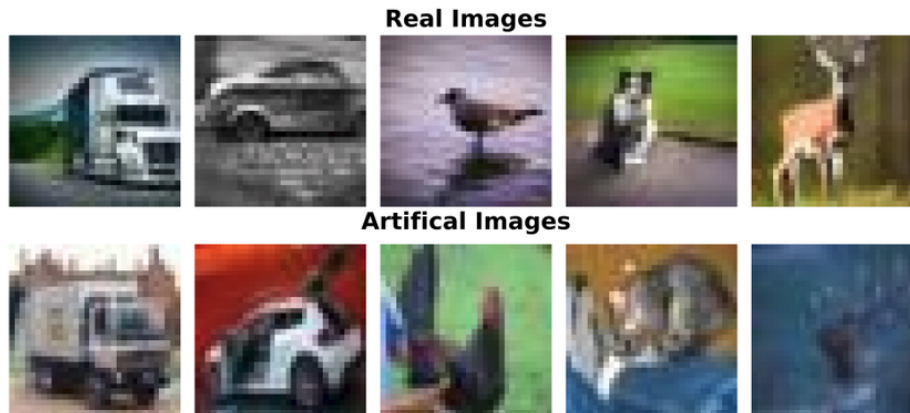


Figure 12: Examples from the CIFAKE dataset

#### 6.1.1.2 Preprocessing

All images are resized to  $224 + \frac{224}{8}$  and then center-cropped to a fixed resolution of  $224 \times 224$ . The images are converted into tensor format without applying normalization at this stage, as normalization is handled later in the model pipeline.

In addition to RGB inputs, augmented feature channels are optionally extracted from the grayscale version of the image. Specifically, Fast Fourier Transform (FFT) features are computed to capture frequency-domain artifacts, and Local Binary Pattern (LBP) features are extracted to encode local texture inconsistencies. These feature maps are concatenated with the original RGB channels to form a multi-channel input representation.

## 6.1.2 COCOFake Datasets

### 6.1.2.1 Overall Information

We conduct experiments on the **COCOFake dataset**, which extends the COCO dataset by incorporating both real and AI-generated images. Each sample is associated with semantic captions, and synthetic images are generated based on these captions. Here are some more detailed information:

- **Number of samples:** The dataset contains a large-scale collection of images, where each real image is paired with 5 synthetic images generated using Stable Diffusion. This results in a dataset with 710,000 samples
- **Training & Validation & Testing dataset:** The dataset comprises over 650,000 training images, 30,000 validation images, and 30,000 test images, following the original splits of the COCO dataset.
- **Image size:** Images have varying resolutions as in the original COCO dataset.

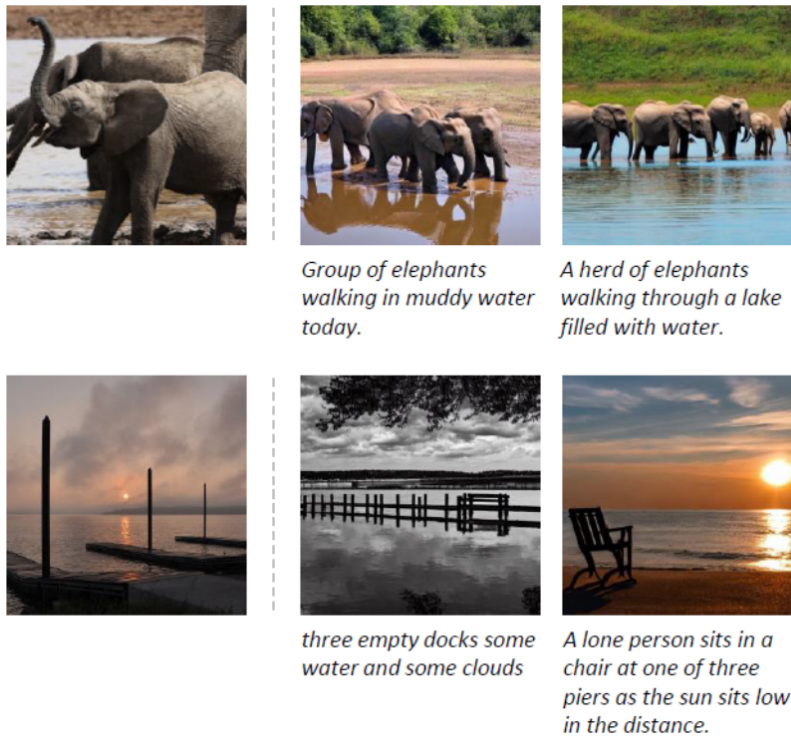


Figure 13: Examples from the COCOFake dataset

### 6.1.2.2 Preprocessing

All images are resized such that the longest dimension is 252 pixels, then center-cropped to  $224 \times 224$ . Training images use random crops and data augmentation: random horizontal/vertical flips, rotations ( $90^\circ$  or  $270^\circ$ ), and color jitter (0–20%). Normalization is applied inside the model (ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225]) after the Adapter, maintaining consistency with the backbone’s pre-training. Augmented feature channels (FFT, LBP) are computed from the grayscale version of each image during the forward pass.

## 6.2 Experimental Setup

We use a pretrained BNext backbone, with experiments conducted on multiple model sizes (Tiny, Small, Medium). The backbone can be either frozen or fine-tuned during training.

All experiments are conducted using a unified training configuration to ensure fairness and reproducibility. The detailed setup, including model configurations, training hyperparameters, and hardware specifications, is summarized following:

Table 1: Experimental configuration for the proposed method.

| Aspect                 | Category              | Setting                                         |
|------------------------|-----------------------|-------------------------------------------------|
| Model                  | Backbone              | BNext-T (29.8M)                                 |
|                        |                       | BNext-S (67.1M)                                 |
| Training Configuration | Backbone Training     | BNext-M (133M)                                  |
|                        |                       | Unfrozen/Frozen                                 |
|                        | Batch Size            | 32                                              |
|                        | Gradient Accumulation | 4 steps                                         |
|                        | Epochs                | 10 (CIFAKE)    6 (COCOFake)                     |
|                        | Optimizer             | AdamW                                           |
|                        | Learning Rate         | $1 \times 10^{-4} \rightarrow 1 \times 10^{-5}$ |
|                        | Mixed Precision       | 16-bit (FP16)                                   |
| System Hardware        | Gradient clipping     | Norm-based, val = 1.0                           |
|                        | GPU Type              | NVIDIA GeForce RTX 4060 Ti (16GB)               |
|                        | CPU                   | 6 cores / 12 logical                            |

## 6.3 Results & Comparison

We evaluate the performance of the proposed method using two standard classification metrics, namely **Accuracy** and **Area Under the ROC Curve (AUC)**.

Table 2: Comparison between reported results and our reproduced results on CIFAKE.

| Source    | Model            | Accuracy     | AUC          | Parameters (M) | FLOPs (G) |
|-----------|------------------|--------------|--------------|----------------|-----------|
| Paper     | BNext-T (frozen) | 83.89        | 91.70        | 29.8           | 0.89      |
| Paper     | BNext-S (frozen) | 80.71        | 89.25        | 67.1           | 1.91      |
| Paper     | BNext-M (frozen) | 82.77        | 90.73        | 133            | 3.39      |
| Paper     | BNext-T          | 97.29        | <u>99.65</u> | 29.8           | 0.89      |
| Paper     | BNext-S          | 96.96        | 99.55        | 67.1           | 1.91      |
| Paper     | BNext-M          | <u>97.35</u> | 99.62        | 133            | 3.39      |
| Retrained | BNext-T (frozen) | 83.66        | 91.68        | 29.8           | 4.86      |
| Retrained | BNext-S (frozen) | 83.82        | 91.63        | 67.1           | 10.91     |
| Retrained | BNext-M (frozen) | 82.27        | 90.59        | 133            | 20.18     |
| Retrained | BNext-T          | 97.31        | 99.64        | 29.8           | 4.86      |
| Retrained | BNext-S          | 97.46        | <u>99.65</u> | 67.1           | 10.91     |
| Retrained | BNext-M          | <b>97.50</b> | <b>99.66</b> | 133            | 20.18     |

Table 3: Comparison between reported results and our reproduced results on COCOFake.

| Source    | Model            | Accuracy     | AUC          | Parameters (M) | FLOPs (G) |
|-----------|------------------|--------------|--------------|----------------|-----------|
| Paper     | BNext-T (frozen) | 83.65        | 81.98        | 29.8           | 0.89      |
| Paper     | BNext-S (frozen) | 93.15        | 95.19        | 67.1           | 1.91      |
| Paper     | BNext-M (frozen) | 84.59        | 82.11        | 133            | 3.39      |
| Paper     | BNext-T          | 99.25        | 99.86        | 29.8           | 0.89      |
| Paper     | BNext-S          | 99.28        | 99.89        | 67.1           | 1.91      |
| Paper     | BNext-M          | 99.18        | <u>99.91</u> | 133            | 3.39      |
| Retrained | BNext-T (frozen) | 83.42        | 81.85        | 29.8           | 4.86      |
| Retrained | BNext-S (frozen) | 93.38        | 95.27        | 67.1           | 10.91     |
| Retrained | BNext-M (frozen) | 84.25        | 81.92        | 133            | 20.18     |
| Retrained | BNext-T          | 99.21        | 99.84        | 29.8           | 4.86      |
| Retrained | BNext-S          | <u>99.34</u> | 99.90        | 67.1           | 10.91     |
| Retrained | BNext-M          | <b>99.45</b> | <b>99.93</b> | 133            | 20.18     |

## 6.4 Experimental Result Analysis

Overall, the reproduced results closely match the performance reported in the original paper, with only minor discrepancies observed across most configurations.

- **Frozen backbone:** Differences in accuracy are generally within a small margin (approximately 1–3%), indicating that the implementation is consistent with the original method.
- **Trainable backbone:** Reproduced results are highly competitive and, in some cases, slightly outperform the reported results.
- **FLOPs difference:** The FLOPs in our reproduced results are higher than those reported in the paper.

The observed discrepancies can be attributed to several factors:

- **Hardware difference:** The experiments are conducted on a different GPU architecture (RTX 4060 Ti) compared to the one used in the original paper (RTX 2080 Ti). Differences in CUDA kernels, Tensor Core behavior, and mixed precision execution can lead to minor numerical discrepancies that accumulate during training.
- **Reproducibility and randomness:** Deep learning training is inherently stochastic. Even with a fixed random seed, factors such as data loading order, parallel workers, and CUDA non-determinism can introduce slight variations in results.
- **Theoretical vs. measured FLOPs:** The FLOPs reported in the original paper are theoretical estimates computed under idealized assumptions for a single forward pass. In contrast, the FLOPs measured in our experiments reflect the actual computational cost during training, which includes additional operations such as backward propagation, mixed precision scaling, and data-dependent overhead.

## CHAPTER 7

### EXTENSION & IMPROVEMENT

#### 7.1 Motivation

The original paper uses BNext as the sole backbone option. However, even though BNEXT is computationally efficient as a binary network, it still remains relatively large (29.8M–133M parameters) and its theoretical speedup requires specialized hardware accelerators (for XNOR operations) not commonly available on commercial devices. To bridge the gap between theoretical efficiency and practical real-time deployment, we propose a novel architecture named **MobileNet4DFR**.

This architecture integrates three complementary improvements to further enhance efficiency and feature fusion quality:

- **Alternative lightweight backbone:** MobileNetV3-Small, which is a standard full-precision efficient network with proven deployment records on mobile hardware.
- **Enhanced Adapter with Squeeze-and-Excitation (SE) attention:** Replacing the single convolution Adapter with a multi-stage module that applies channel-wise attention before dimensionality reduction.
- **Gabor filter channel:** Replacing the Sobel magnitude channel with a Gabor filter response, providing richer directional frequency information to capture microscopic manipulation traces.

#### 7.2 Proposed Architecture: MobileNet4DFR

The proposed improvements were implemented directly by modifying the original architecture:

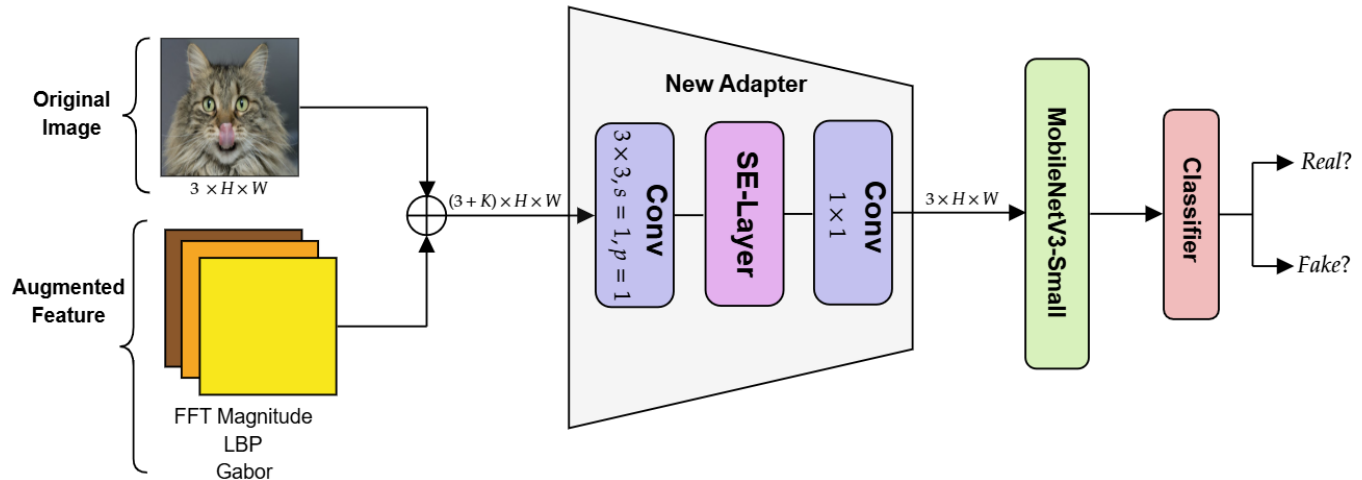


Figure 14: Proposed Architecture

### 7.2.1 Backbone Replacement with MobileNetV3-Small

While BNext reduces computational cost via quantization, its parameter count (29.8M for the Tiny version) still poses memory bottleneck challenges. We replace the BNext backbone with MobileNetV3-Small [10].

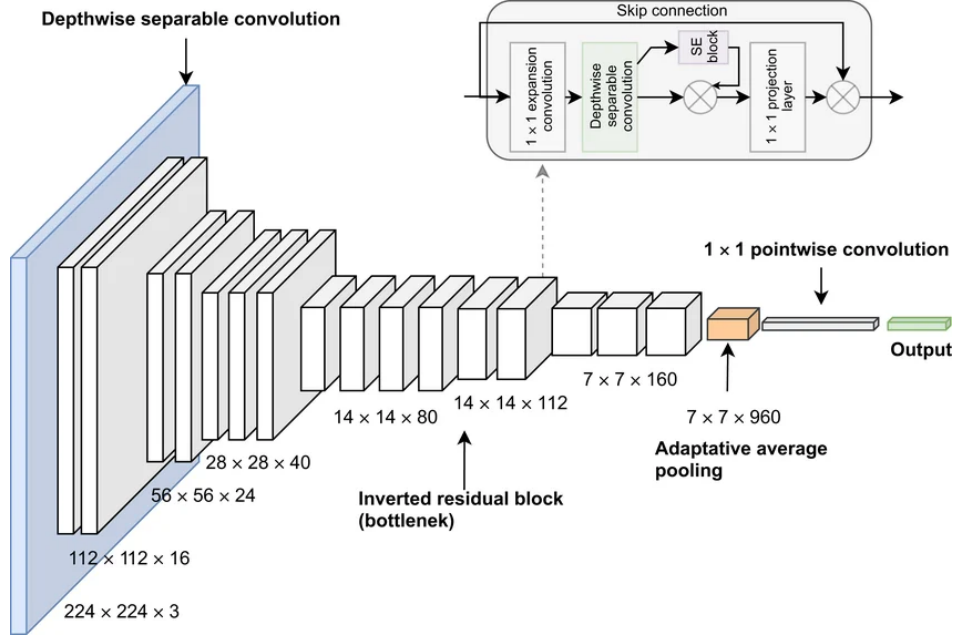


Figure 15: MobileNetV3 Architecture

MobileNetV3-Small is designed for efficiency and lightweight deployment. It significantly reduces the number of parameters to around **2.5 million**, which is much smaller compared to BNext-T. This makes the model more suitable for real-time applications and resource-constrained environments.

In addition, MobileNetV3-Small is well-supported by existing deep learning frameworks and hardware accelerators, allowing it to run efficiently on common GPUs and CPUs without requiring specialized optimizations. While it does not rely on extreme model compression techniques like BNext, it achieves a strong balance between performance and efficiency in practice.

### 7.2.2 Enhanced Adapter with Squeeze-and-Excitation (SE) Block

The original Adapter is a single convolutional layer that treats all input channels equally when compressing from 5 (or more) channels to 3. This is suboptimal: the RGB, FFT, Gabor and LBP channels carry qualitatively different information and should be weighted differently.

To improve this, we introduce a **Squeeze-and-Excitation (SE) Block** into the Adapter. The SE block operates in two phases:

- **Squeeze:** Global Average Pooling computes a channel descriptor  $y \in \mathbb{R}^C$  by averaging spatial information:

$$y_c = \frac{1}{H \cdot W} \sum_{i,j} x_c(i,j)$$

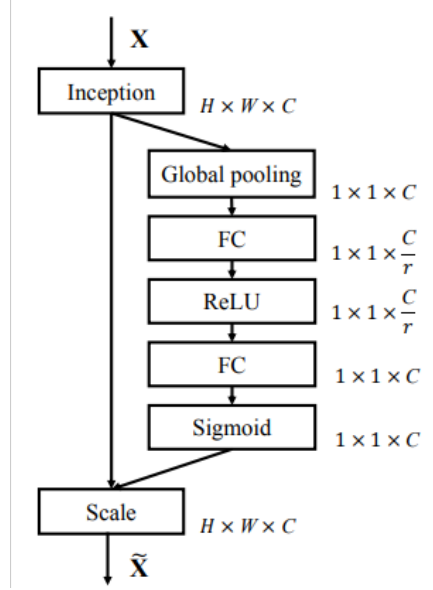


Figure 16: SE Block

- **Excitation:** Two fully connected layers with a bottleneck (reduction ratio = 16):  $\text{Linear}(C, C/16) \rightarrow \text{ReLU} \rightarrow \text{Linear}(C/16, C) \rightarrow \text{Sigmoid}$ , producing channel-wise attention weights  $s \in [0, 1]^C$ .
- **Scale:** The input is re-weighted:

$$\hat{x}_c = s_c \cdot x_c$$

emphasizing informative channels and suppressing less useful ones.

The SE block is applied after the first convolution (which mixes spatial information across the  $3+N$  channels) but before the final  $1 \times 1$  convolution (which performs the final channel compression). This allows the network to selectively amplify, for example, the LBP channel when texture anomalies are the dominant cue, or the FFT channel when frequency artifacts are more salient.

Also, the original paper notes that the Adapter must “preserve the most informative cues from the augmented representation” — but a plain convolution applies fixed weights regardless of input content. The SE mechanism makes this preservation input-adaptive and content-aware.

### 7.2.3 Gabor Filter Channel Augmentation

Generative AI models often leave structural artifacts around high-frequency regions with specific orientations, such as eyelashes, hair strands, and wrinkles. While LBP captures local isotropic textures, it lacks orientation selectivity. We augment the spatial feature extraction by introducing a **Gabor Filter Bank**.

In the ablation study, the Sobel magnitude channel is replaced by a Gabor filter response. The image is convolved with Gabor kernels at four orientations ( $\theta \in \{0, \frac{\pi}{4}, \frac{\pi}{2}, \frac{3\pi}{4}\}$ ). To maintain a compact input tensor, we compute the maximum response across all orientations at each spatial location:

$$G_{max}(x, y) = \max_{\theta} |I(x, y) * \psi(x, y; \lambda, \theta)| \quad (1)$$

### 7.3 Ablation Study Analysis & Discussion

Following the paper’s methodology, we conduct an ablation study to evaluate the contribution of each feature channel in the improved architecture. We replace the Sobel magnitude channel with Gabor and test all combinations:

Table 4: Ablation study configurations for the MobileNet4DFR (improved model)

| Configuration       | Channels                | Expected behavior              |
|---------------------|-------------------------|--------------------------------|
| Baseline (RGB only) | RGB                     | No augmented features          |
| FFT only            | RGB + FFT               | Spectral fingerprint           |
| Gabor only          | RGB + Gabor             | Directional frequency features |
| LBP only            | RGB + LBP               | Local texture encoding         |
| FFT + Gabor         | RGB + FFT + Gabor       | Dual frequency-domain          |
| FFT + LBP           | RGB + FFT + LBP         | Paper’s best config            |
| Gabor + LBP         | RGB + Gabor + LBP       | Spatial + directional texture  |
| FFT + Gabor + LBP   | RGB + FFT + Gabor + LBP | Full augmentation              |

Table 5: Ablation study on the CIFAKE Dataset.

| Ablation                           | Variations         | Accuracy (%) |
|------------------------------------|--------------------|--------------|
| Baseline                           | –                  | <u>95.81</u> |
| Features added to the learned ones | FFT                | 96.44        |
|                                    | Gabor              | 96.17        |
|                                    | LBP                | 96.88        |
|                                    | FFT and Gabor      | 96.72        |
|                                    | <b>FFT and LBP</b> | <b>97.93</b> |
|                                    | Gabor and LBP      | 97.49        |
|                                    | FFT, Gabor and LBP | 97.15        |

Based on the empirical results presented in Table 5, we can draw several critical insights regarding the behavior of multi-modal feature fusion in deepfake detection:

The baseline model using only RGB inputs achieves 95.81% accuracy. Among individual features, LBP performs best, indicating that local texture inconsistencies are particularly effective for detecting diffusion-generated images. Combining features from different domains leads to further improvements, with FFT + LBP achieving the highest accuracy (97.93%), as they provide complementary spatial and frequency information.

In contrast, combinations involving Gabor do not yield the best results due to overlapping information with either FFT or LBP. Moreover, using all features together slightly degrades performance (97.15).

Overall, the results confirm that selecting complementary features is more effective than simply increasing the number of input features, with RGB + FFT + LBP emerging as the optimal configuration.

## 7.4 Experimental Comparison

Table 6: Comparison of proposed improvements vs. paper baselines and SOTA on CIFAKE

| Method | Model                        | Accuracy     | AUC          | Parameters (M) | FLOPs (G)    |
|--------|------------------------------|--------------|--------------|----------------|--------------|
| [12]   | ResNet-50                    | 95.00        | 99.00        | 25.6           | 4.8          |
|        | VGG                          | 96.00        | 99.00        | 133            | 7.63         |
|        | DenseNet                     | <b>98.00</b> | 99.00        | <u>7.9</u>     | 5.6          |
| Paper  | BNext-T with frozen backbone | 83.89        | 91.70        | 29.8           | <u>0.89</u>  |
|        | BNext-S with frozen backbone | 80.71        | 89.25        | 67.1           | 1.91         |
|        | BNext-M with frozen backbone | 82.77        | 90.73        | 133            | 3.39         |
|        | BNext-T                      | 97.29        | <u>99.65</u> | 29.8           | <u>0.89</u>  |
|        | BNext-S                      | 96.96        | 99.55        | 67.1           | 1.91         |
|        | BNext-M                      | 97.35        | 99.62        | 133            | 3.39         |
| Ours   | <b>MobileNet4DFR</b>         | <u>97.93</u> | <b>99.71</b> | <b>1.52</b>    | <b>0.070</b> |

Table 7: Comparison of proposed improvements vs. paper baselines and SOTA on COCOFake

| Method | Model                        | Accuracy     | AUC          | Parameters (M) | FLOPs (G)    |
|--------|------------------------------|--------------|--------------|----------------|--------------|
| [13]   | ResNet50                     | 90.31        | —            | <u>25.6</u>    | 4.8          |
|        | ViT-B/32                     | 87.64        | —            | 88.3           | 8.56         |
|        | CLIP-ResNet50                | 99.07        | —            | 25.6           | 4.8          |
|        | CLIP-ViT-B/32                | 99.11        | —            | 88.3           | 8.56         |
|        | OpenCLIP-ViT-B/32            | 97.88        | —            | 88.3           | 8.56         |
|        | OpenCLIP-ViT-B/32            | <b>99.68</b> | —            | 88.3           | 8.56         |
| Paper  | BNext-T with frozen backbone | 83.65        | 81.98        | 29.8           | <u>0.89</u>  |
|        | BNext-S with frozen backbone | 93.15        | 95.19        | 67.1           | 1.91         |
|        | BNext-M with frozen backbone | 84.59        | 82.11        | 133            | 3.39         |
|        | BNext-T                      | 99.25        | 99.86        | 29.8           | <u>0.89</u>  |
|        | BNext-S                      | <u>99.28</u> | <u>99.89</u> | 67.1           | 1.91         |
|        | BNext-M                      | 99.18        | <b>99.91</b> | 133            | 3.39         |
| Ours   | <b>MobileNet4DFR</b>         | 98.07        | 99.57        | <b>1.52</b>    | <b>0.070</b> |

Based on the results in Tables 4 and 5, **MobileNet4DFR** achieves a strong balance between performance and efficiency.

On CIFAKE, the model reaches **97.93% Accuracy** and **99.71% AUC**, outperforming all BNext variants and slightly surpassing recent SOTA in AUC, while using significantly fewer parameters and FLOPs.

On the more challenging COCOFake dataset, it achieves **98.07% Accuracy** and **99.57% AUC**. Although slightly below large-scale models, the performance gap is minimal compared to the substantial reduction in computational cost.

The key advantage lies in efficiency: **MobileNet4DFR** uses only **1.52M parameters** and **0.070G FLOPs**, which is dramatically lower than both BNext and transformer-based models. Unlike BNext, its computations rely on standard operations, making it practical for real-time deployment on common hardware.

Overall, MobileNet4DFR demonstrates that a lightweight architecture, when combined with effective feature augmentation and attention mechanisms, can achieve competitive performance while being highly efficient and deployable.

## CHAPTER 8

### CONCLUSION

#### 8.1 Derivation from Course Theory

The paper builds directly upon the LBP theory established in Chapter 2. Specifically, LBP serves as an auxiliary texture channel feeding into a deep neural network—the descriptor remains identical in its core formulation (pixel-wise neighborhood comparison producing binary codes), but is utilized as a spatial image channel rather than a globally pooled histogram. The authors’ rationale—that deepfakes typically struggle to maintain micro-structural accuracy in facial texture regions—mirrors exactly the discriminative power that makes LBP a classical powerhouse for face recognition.

Similarly, the FFT magnitude channel parallels the Gabor filter framework: both operate in the frequency domain to expose structured patterns that are invisible in the spatial domain. While Gabor provides localized, multi-scale frequency analysis, FFT provides a global spectral profile. The paper implicitly leverages the identical theoretical insight but opts for a computationally simpler implementation.

#### 8.2 Methodological Differences from Classical Approaches

- **Integration Paradigm:** Classical LBP systems utilize handcrafted histogram features fed into shallow classifiers (e.g., k-NN, SVM). Conversely, the paper integrates LBP as a raw CNN input channel, empowering the network to learn data-driven representations of texture, frequency, and color.
- **Binary Backbone:** BNNs replace floating-point multiplications with XNOR and POP-COUNT operations—a paradigm shift entirely absent from classical LBP pipelines.
- **Adapter Layer:** There is no direct analogue in classical descriptor pipelines. The Adapter acts as a dimensional bridge between the augmented multi-channel input (RGB + FFT + LBP) and the pre-trained backbone’s rigid 3-channel expectation.
- **Task Domain:** Classical LBP was heavily optimized for identity and expression recognition (multi-class mapping); the paper pivots its application to binary real/fake classification—a fundamentally different objective focusing on generative artifacts rather than semantic identity.

#### 8.3 Technical Strengths

- **Efficiency without Accuracy Sacrifice:** BNext-T (0.89 GFLOPs) achieves an impressive 99.25% on COCOFake, outperforming ResNet-50’s 90.31% which requires 4.8 GFLOPs.
- **Principled Feature Augmentation:** LBP and FFT address strictly complementary domains (spatial texture vs. global frequency). The ablation study rigorously confirms their synergy.

- **Comprehensive Evaluation:** Validating the model across three diverse datasets covering both diffusion models (COCOFake, CIFAKE) and GANs (DFFD) demonstrates robust generalizability.

## 8.4 Limitations & Discrepancies

- **Theoretical vs. Practical Speedup:** The heavily advertised  $58\times$  inference speedup mandates specialized XNOR/POPCOUNT hardware accelerators. On standard commercial GPUs, the actual wall-clock speedup is significantly marginalized.
- **Code vs. Paper Discrepancy (LBP Configuration):** The paper claims the use of  $LBP_{8,1}$ . However, upon inspecting the official source code, the implementation actually utilizes  $P = 3, R = 6$  with the uniform method (`skimage.feature.local_binary_pattern(gray, 3, 6, method='uniform')`). This discrepancy highlights a gap between the manuscript and the reproducible codebase.
- **Lack of Cross-Dataset Evaluation:** The paper strictly trains and tests within the same dataset domains. It fails to evaluate zero-shot generalization (e.g., training on CIFAKE and testing on DFFD) to entirely unseen generation techniques.

## 8.5 Personal Technical Insights & Conclusion

The most instructive finding from the ablation study is the phenomenon of feature synergy: neither FFT nor LBP alone could outperform the naive RGB baseline. Yet, when fused, they exceeded the baseline by 1.25%. This confirms the theoretical prediction from Chapter 2 that spatial texture features and frequency features are orthogonal and strictly complementary. Furthermore, the Sobel filter’s failure to contribute additional accuracy reinforces the observation that LBP inherently captures edge-like micro-patterns through its neighborhood comparison—rendering standard edge detectors redundant.

Ultimately, there is a conceptually elegant resonance between classical LBP theory and modern BNNs: **both systems fundamentally rely on binarizing their respective signals using the sign function.** LBP binarizes local pixel-neighborhood relationships to extract texture; BNNs binarize global network weights and activations to extract computational efficiency.

By successfully reproducing the paper’s results, deeply analyzing the source code to find implementation discrepancies, and proposing the **MobileNet4DFR** architecture to solve the practical deployment bottleneck, our team has thoroughly bridged the gap between the foundational theories of Pattern Recognition and state-of-the-art Deep Learning applications.

## References

- [1] Lanzino, R., Fontana, F., Diko, A., Marini, M. R., & Cinque, L. (2024). *Faster Than Lies: Real-time Deepfake Detection using Binary Neural Networks*. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW 2024)* (pp. 3771–3780).
- [2] Ojala, T., Pietikäinen, M., & Mäenpää, T. (2002). Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7), 971–987.
- [3] Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face description with local binary patterns: Application to face recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12), 2037–2041.
- [4] Zhao, G., & Pietikäinen, M. (2007). Dynamic texture recognition using local binary patterns with an application to facial expressions. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6), 915–928.
- [5] Daugman, J. G. (1985). Uncertainty relation for resolution in space, spatial frequency, and orientation optimized by two-dimensional visual cortical filters. *Journal of the Optical Society of America A*, 2(7), 1160–1169.
- [6] Wiskott, L., Fellous, J. M., Krüger, N., & von der Malsburg, C. (1997). Face recognition by elastic bunch graph matching. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(7), 775–779.
- [7] Guo, N., Bethge, J., Meinel, C., & Yang, H. (2022). Join the high accuracy club on ImageNet with a binary neural network ticket. *arXiv preprint arXiv:2211.12933*.
- [8] Courbariaux, M., Hubara, I., Soudry, D., El-Yaniv, R., & Bengio, Y. (2016). Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *arXiv preprint arXiv:1602.02830*.
- [9] Guo, N., Bethge, J., Meinel, C., & Yang, H. (2022). Join the high accuracy club on ImageNet with a binary neural network ticket. *arXiv preprint arXiv:2211.12933*.
- [10] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., & Vasudevan, V. (2019). Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 1314–1324).
- [11] Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 7132–7141).
- [12] Wang, Y., Hao, Y., & Cong, A. X. (2024). Harnessing machine learning for discerning AI-generated synthetic images. *arXiv preprint arXiv:2401.07358*.
- [13] Amoroso, R., Morelli, D., Cornia, M., Baraldi, L., Del Bimbo, A., & Cucchiara, R. (2024). Parents and children: Distinguishing multimodal deepfakes from natural images. *ACM Transactions on Multimedia Computing, Communications and Applications*, 21(1), 1–23.